Cursor Prompt — Aurum DQA Monitoring Platform (Full Build)

You are building a data quality monitoring platform for The Aurum Institute, replacing an entirely Excel-based process. This is a single, complete build — every feature below is in scope. Read this whole prompt before writing any code.

Context

Data Quality Monitors (DQMs) audit TB/DS-TB data quality per facility, per month, in an Excel workbook (`ACC1_DS-TB_DQA_tool_v3.xlsx`), comparing register/tally-sheet, summary sheet, TIER.Net, and DHIS data — Before and After a facility visit — across age bands and indicator categories (TB cascade, TPT, DSTB/DRTB outcomes, register-based lists). Each DQM drops their completed facility file into a shared Teams folder each month. A MERL Officer manually consolidates every facility's file into one dataset and further transforms it before loading it into Power BI for management in another office.

This platform removes the manual consolidation and transformation labor, gives DQMs and the MERL Officer a way to monitor data quality trends over time (monthly/quarterly/yearly — not just one isolated month), and produces the Power BI-ready export automatically.

Stack

- **Frontend/backend**: Next.js (App Router), deployed to Vercel
- **Database**: Postgres on Neon
- **ORM**: Drizzle or Prisma — pick one and use it consistently throughout
- **Auth**: role-based auth (`dqm`, `merl_officer`) — NextAuth with credentials or magic link is fine
- **Styling**: Tailwind
- **Excel parsing**: `exceljs` or `xlsx` (SheetJS) on the Node side

1. Database schema

```
create table facilities (
  id serial primary key,
  name text not null unique,
  district text,
  sub_district text
```

```
);

create table users (
  id serial primary key,
  name text not null,
  email text unique not null,
  role text not null check (role in ('dqm', 'merl_officer'))
);

create table entries (
  id bigserial primary key,
  facility_id int not null references facilities(id),
  period_date date not null,        -- first day of the reporting month
  data_type text not null,          -- e.g. 'TB cascade', 'TPT', 'DSTB outcome', 'DRTB outcome'
  age_group text not null,          -- 'Under 5yrs', 'Over 5yrs', 'All ages'
  indicator text not null,          -- e.g. 'Headcount', 'TB screening'
  source text not null,             -- 'register', 'summary_sheet', 'tier_net', 'dhis'
  stage text not null check (stage in ('before', 'after')),
  value numeric not null,
  entry_method text not null check (entry_method in ('web_form', 'excel_upload')),
  captured_by int references users(id),
  captured_at timestamptz not null default now()
);
create index on entries (facility_id, period_date);

create table facility_month_status (
  facility_id int not null references facilities(id),
  period_date date not null,
  status text not null default 'submitted' check (status in ('submitted', 'reviewed_locked',
'exported')),
  locked_by int references users(id),
  locked_at timestamptz,
  primary key (facility_id, period_date)
);

create table audit_log (
  id bigserial primary key,
  entity text not null,             -- 'entry', 'facility_month_status'
  entity_id text not null,
  action text not null,             -- 'create', 'edit', 'lock', 'unlock', 'comment'
  performed_by int references users(id),
  performed_at timestamptz not null default now(),
  detail jsonb                      -- before/after values, comment text, etc.
```

);

Rates (screening rate, treatment success rate, LTFU rate, etc.) are NEVER stored as their own rows and NEVER averaged across periods. They're always computed from summed underlying counts at query time, at whichever grain is being viewed — build this as a rule the rollup views enforce (see section 4), not something left to whoever writes a query later.

2. Data entry — both paths write into the same `entries` table

2a. Web form

A form where a DQM selects facility + reporting month, then enters Before/After values per indicator × age group × source. Build the indicator list as a configurable list (JSON/config file), not hardcoded, since the real indicator set spans many categories (TB cascade, TPT, DSTB/DRTB outcomes, several register-based lists). Tag rows `entry_method = 'web_form'`.

2b. Excel upload

DQMs can instead upload their completed `ACC1_DS-TB_DQA_tool_v3.xlsx` file. Build the parser against the actual workbook structure — **do not guess at exact cell coordinates**. At parse time:

- Read the `Before` and `After` sheets, using header rows to map columns to `data_type` / `age_group` / `indicator` / `source`, rather than hardcoding fixed cell references (the current tool's fragility comes from exactly that pattern — don't reproduce it).
- Place a copy of `ACC1_DS-TB_DQA_tool_v3.xlsx` in the repo (e.g. `reference/`) as ground truth for the parser to be developed and tested against.
- Each row parsed becomes one `entries` row with `entry_method = 'excel_upload'`.
- On successful parse, create/update the `facility_month_status` row to `submitted`, same as the web form path.
- Validate on upload: if Before/After sources for an indicator disagree, don't silently accept it — surface it clearly in the UI as a flagged mismatch (this replaces the current `"data is correct"` / `"correct data"` formulas, which are easy to misread under time pressure — make the flagged state visually unambiguous, e.g. a red banner naming the actual discrepancy, not a string that looks like its own opposite).

3. Review & lock workflow (MERL Officer)

- A screen listing all facility-months with status `submitted` for a given month, with the entered data and any flagged mismatches visible per facility.
- A "Lock" action sets `status = 'reviewed_locked'`, records `locked_by`/`locked_at`, and writes an `audit_log` entry.
- Locked facility-months are read-only through the UI, enforced in the API layer.
- **Auto-lock**: a month locks automatically after the 10th of the following month if not already locked manually. Implement this as a scheduled job (Vercel Cron or similar).
- Corrections to an already-locked month are NOT made by reopening it. Instead, build a "correction" action available in the *current* month's entry flow that references the prior month/facility/indicator being corrected, and writes both a new `entries` row (tagged appropriately) and an `audit_log` entry — history is never rewritten in place.

4. Trend views — monthly, quarterly, yearly

Build as SQL views (or equivalent query functions), all deriving from `entries`, never from each other's stored output:

```
create view v_monthly as
select facility_id, date_trunc('month', period_date)::date as period,
    data_type, age_group, indicator, source, stage, sum(value) as value
from entries
group by 1,2,3,4,5,6,7;

create view v_quarterly as
select facility_id, date_trunc('quarter', period_date)::date as period,
    data_type, age_group, indicator, source, stage, sum(value) as value
from entries
group by 1,2,3,4,5,6,7;

create view v_yearly as
select facility_id, date_trunc('year', period_date)::date as period,
    data_type, age_group, indicator, source, stage, sum(value) as value
from entries
group by 1,2,3,4,5,6,7;
```

Rate indicators (e.g. `TB screening rate = TB screening / Headcount`) are computed on top of these views by joining numerator/denominator indicators and dividing — build this as a small, reusable function/query layer so the same rate logic runs correctly at all three grains.

Build a dashboard with:

- Trend lines per facility per indicator across the selected grain (toggle month/quarter/year)
- Before/After deltas per visit
- Cross-facility comparison at a given period (which facilities are flagged this month)
- Source-agreement rate over time (% of indicators where register/TIER.Net/DHIS already agreed before the visit, trended over time)

No facility-level access restriction — any authenticated user (DQM or MERL Officer) can view all facilities.

5. Power BI export

Generate a file (CSV or XLSX) matching the long-format shape of the current `Power Query` tab: `Activity, TB type, Dataset, Staff name, Date of visit, District, Sub-district, Facility Name, Authority, Data type, Age category, Element, source columns (register/summary/TIER.Net/DHIS), Comments`, sourced only from `facility_month_status.status = 'reviewed_locked'` months. Build this as:

- An on-demand "Export" action for the MERL Officer, scoped to a selected month
- A scheduled job that runs after each auto-lock, generating the file automatically

Build the export against this known column shape now. Add a clearly-marked, isolated transformation step (a single function, not scattered logic) for whatever adjustment the MERL Officer currently applies before sending to management — leave this function as a documented pass-through (`applyMerlTransformation(rows)`) that returns rows unchanged for now, so it's a single, obvious place to add the real logic once it's confirmed against an actual sample of a `Power Query` tab and its matching sent file for the same month. Don't scatter guessed transformation logic across the export — isolate the unknown to one function.

6. Historical backfill

Build an import script (not a UI feature) that parses prior months' `ACC1_DS-TB_DQA_tool_v3.xlsx` files using the same parser as section 2b, targeting **up to 24 months back**, per the look-back rules already defined in the tool's own `Instructions` tab:

- General DQA data: 2 months back from each file's reporting point
- Discharge / outstanding outcomes lists: effectively ~7 months back (2 + 5)
- DSTB cohort outcomes: 12 months back
- DRTB cohort outcomes: 24 months back (the deepest requirement, hence the 24-month backfill target)

Backfilled facility-months should be created directly in `reviewed_locked` status (they're historical and already actioned), not `submitted`.

7. Audit trail

Every `entries` insert, every `facility_month_status` transition, and every correction gets a row in `audit_log`. Build a simple audit view per facility-month so a MERL Officer can see the full history of what happened, by whom, and when — something the current Excel process cannot show at all.

Deliverable

A running Next.js app on Vercel, schema migrated on Neon, with: web form entry, Excel upload/parsing, the review/lock workflow with auto-lock after the 10th, monthly/quarterly/yearly trend views with correct rate handling, the Power BI export (with the MERL transformation function isolated and pass-through for now), the historical backfill script, and a full audit log. Seed the database with a handful of facilities and a few months of sample data so the dashboard has something to show immediately.